

## Part02

**2- What's Enum data type, when is it used? And name three common built-in enums used frequently?**

**Enum (Enumeration)** is a data type in C# that is used to define a set of **named constant values**.

**When is it used?**

It is used when a variable should have a **limited number of possible values**, making the code easier to read and understand.

**Example:**

```
enum Days
{
    Saturday,
    Sunday,
    Monday
}
```

**Three common built-in enums:**

1. DayOfWeek
2. DateTimeKind
3. StringComparison

**Short answer for exam:**

Enum is a value type used to define a set of named constants. It is useful when a variable has a limited set of possible values. Examples include DayOfWeek, DateTimeKind, and StringComparison.

### 3- What are scenarios to use string vs StringBuilder?

#### Use string when:

- You have a small number of string operations.
- The string doesn't change frequently.
- You are mainly reading or comparing text.

#### Use StringBuilder when:

- You need to modify a string **many times**.
- You are repeatedly adding, removing, or changing characters.
- You want to reduce the creation of multiple string objects in memory.

#### Example:

```
string name = "Mohamed";  
name += " Desoky";    // Few changes → string is fine
```

```
StringBuilder sb = new StringBuilder();  
  
for (int i = 0; i < 1000; i++)  
{  
    sb.Append(i);  
}
```

#### Short answer for exam:

Use string for simple text operations and when the value doesn't change frequently. Use StringBuilder when performing many modifications to a string, especially inside loops, because it is more efficient for repeated changes.

## Part03 Bonus

### 5- What is meant by User-Defined Constructor and its role in initialization?

A **User-Defined Constructor** is a constructor that is created by the programmer to initialize an object with specific values when it is created.

#### Example:

```
class Student
{
    public string Name;
    public int Age;

    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

When we create the object:

```
Student s = new Student("Mohamed", 20);
```

The constructor automatically runs and initializes Name and Age.

#### Short answer:

A user-defined constructor is a constructor created by the programmer to initialize an object's fields or properties when the object is created.

## 6- Compare between Array and Linked List

### Array

Stores elements in **contiguous memory**

Usually has a **fixed size**

Fast access using index:  **$O(1)$**

Insertion/deletion can be expensive:  **$O(n)$**

Uses less memory per element

Better when frequent access by index is needed

### Linked List

Elements can be stored in **different memory locations**

Can grow or shrink dynamically

Access by index is slower:  **$O(n)$**

Insertion/deletion can be efficient when the node position is known

Requires extra memory for pointers/references

Better when frequent insertion and deletion are needed

### Simple example:

#### Array:

```
int[] numbers = { 10, 20, 30, 40 };
```

#### Linked List:

```
LinkedList<int> numbers = new LinkedList<int>();  
numbers.AddLast(10);  
numbers.AddLast(20);  
numbers.AddLast(30);
```

### Short answer for exam:

An Array stores elements in contiguous memory and provides fast index-based access, while a Linked List stores elements in nodes connected by references and provides more flexibility for insertion and deletion.



I'm looking for...



**Mohamed Desoky** • You

Brand partnership • Full-Stack .NET Web De...

[View my services](#)

1m •

هو أنا كل ما أغير String في C# بعمل String جديدة؟  
في الأول كنت فاكّر إن الـ 'String' عادي جدًا، أغير قيمتها وخلص.  
لكن وأنا بتعلم C# بدأت أفهم حاجة اسمها **Immutability**، وهنا  
الموضوع بدأ يفرق معايا  
ببساطة كده:

الـ String في C# Immutable

يعني إيه؟

يعني لما يكون عندي:

```

csharp
string name = "Mohamed";
name = name + " Desoky";

```

أنا هنا مش بعديل على الـ String القديمة.  
اللي بيحصل إن C# بتعمل String جديدة بالقيمة الجديدة،  
والقديمة بتفضل زي ما هي لحد ما الـ Garbage Collector يتعامل  
معاها.

طيب هنا سؤال مهم...

طب لو أنا بعمل تعديلات كتير على String؟

وهنا بقى بيظهر دور **StringBuilder**

معمول عشان أقدر أعديل على النص بشكل أكثر 'StringBuilder'  
كفاءة، خصوصًا لما أكون بضيف أو بغير حاجات كتير.

مثلاً:

```

csharp
StringBuilder sb = new StringBuilder();
sb.Append("Hello");
sb.Append(" ");
sb.Append("Mohamed");

```

بدل ما أعمل String جديدة مع كل تعديل، الـ 'StringBuilder'  
بيسمحلي أتعامل مع نفس الـ object وأعديل عليه.  
وده خلاني أفهم إن الفرق مش مجرد:

String vs StringBuilder

لكن الفكرة الأهم هي:

- `String` → Immutable
  - `StringBuilder` → Mutable
  - تعديلات كثير على String → ممكن تكون أقل كفاءة
  - تعديلات كثير على النص → `StringBuilder` غالباً اختيار أفضل
- والحقيقة إن دي من الحاجات اللي كل ما بتعلمها بحس إن البرمجة مش مجرد "الكود اشتغل وخلص".
- بقيت أسأل نفسي:
- الكود اشتغل... بس هل أنا كاتبه بطريقة صح وفعالة؟
- أنا لسه بتعلم #C، وكل فترة بحاول أشارك حاجة جديدة بفهمها يمكن تفيد حد لسه بيبدأ زيي.
- لو أنت لسه بتتعلم #C، كنت تعرف الفرق بين String و StringBuilder؟

#DEPI #CSharp #FullStackDotNet

## C# Strings

### Immutability vs Mutability

**STRING (Immutable)** 🔒

```
string name = "Mohamed";  
name = name + ".Desktop";
```

"Mohamed" → "Mohamed Desktop"

(Old string) still in memory      (New string) created

- ✗ Every change creates a new string
- ✗ More memory allocations
- ✗ Slower with frequent changes

**StringBuilder (Mutable)** 🔓

```
StringBuilder sb = new StringBuilder();  
sb.Append("Hello");  
sb.Append(" ");  
sb.Append("Mohamed");
```

"Hello Mohamed"

(Same object) modified

- ✓ Modifies the same object
- ✓ Fewer memory allocations
- ✓ Faster with frequent changes

Which one should I use?

2 impressions

View